Computational Physics: Stochastic Methods and Path Integrals UCSD PHYS 142/242
Assignment 3 Due: Monday, November 16, 2026, 8:00pm

---

## Policies

- You are free to collaborate on all of the problems, subject to the collaboration policy stated in the syllabus.
- You should submit all code used in the homework. You are free to use Python, C/C++, Julia, or any other code **within reason** as long as you comment your code such that the TA can follow along and run it without any issues.
- Please submit your report as a single .pdf file to Gradescope under "Assignment 3". **In the report, include any images generated by your code along with your answers to the questions.** For instructions specifically pertaining to the Gradescope submission process, see `https://www.gradescope.com/get_started#student-submission`.
- Please submit your code and the `results.json` it writes (see the last page) to Gradescope under "Assignment 3 Code", as individual files or a single .zip archive. The assignment is worth 40 points, plus 10 bonus points for Problem 2B. The autograder awards 13 required points and 4 bonus points; the report earns the rest.
- End your report with an AI use appendix, as described in the syllabus: which tools you used and for what, the prompts you used (or a share link), and what the tool got wrong and how you caught it. "I did not use AI tools on this assignment" is a complete answer.
- You will walk through your code for this assignment at Oral Checkpoint 3 (Lab 15, Tuesday, November 17). Bring a laptop that can run it; bonus Problem 2B is not required for the checkpoint.

Computational Physics: Stochastic Methods and Path Integrals UCSD PHYS 142/242
Assignment 3 Due: Monday, November 16, 2026, 8:00pm

---

# 1 MCMC and 2D Ising Model [30 Points]

*Relevant materials: Lectures 14–17, Lab 11*

Consider the 2D Ising model in a square lattice $\Lambda$ with $20 \times 20$ sites and periodic boundary conditions. The energy of the system for a given spin configuration $\sigma = \{\sigma_i\}_{i\in\Lambda}$ is

$$E(\sigma) = -J \sum_{\langle ij \rangle} \sigma_i \sigma_j, \tag{1}$$

where $\langle ij \rangle$ denotes two adjacent sites (with no double counting), $J$ is the spin-spin interaction, and $\sigma_i \in \{-1, +1\}$ is the spin at site $i$.

The total magnetization of the system and the magnetization per spin are

$$M(\sigma) = \sum_{i\in\Lambda} \sigma_i, \qquad m(\sigma) = \frac{M(\sigma)}{|\Lambda|} = \frac{1}{|\Lambda|} \sum_{i\in\Lambda} \sigma_i. \tag{2}$$

The probability of a given configuration $\sigma$ is given by the Boltzmann distribution

$$p(\sigma) = \frac{1}{Z} \exp\left(-\frac{E(\sigma)}{k_B T}\right), \tag{3}$$

where $Z = \sum_\sigma \exp\left(-\frac{E(\sigma)}{k_B T}\right)$ is the partition function, $T$ is the temperature, and $k_B$ is the Boltzmann constant. For this problem, you may set $J = k_B = 1$.

**Problem A [10 points]:** Use MCMC to simulate the 2D Ising model at temperature $T = 2$ and calculate the magnetization per spin $m$ at each time step. Try two initial configurations: one with all spins up and one with spins randomly up or down with equal probability. Plot $m$ as a function of MCMC time steps. What's a good number of "burn-in" steps for the two initial configurations? What's a good number of total steps to get a good estimate of the magnetization?

**Problem B [20 points]:** Re-run the MCMC simulation for the 2D Ising model at temperatures $T = 1$ to 4 in steps of 0.2. Describe your strategy for determining the initial configuration, burn-in steps, number of steps per chain, number of chains, and thinning (if any).

Plot the magnetization per spin $m$ and its standard deviation (as an error bar or band) as a function of temperature. Compare to the theoretical prediction for the critical temperature $T_c = \frac{2}{\ln(1+\sqrt{2})} \approx 2.269$ in units where $J = k_B = 1$. Do you observe a sharp phase transition at the critical temperature? If not, what could be going wrong?

## 2 Path-Integral Monte Carlo at Finite Temperature [10 Points + 10 Bonus Points]

*Relevant materials: Lectures 13 and 16–19, Labs 12 and 14*

In Lab 14, you sampled imaginary-time paths at large $\beta$, where only the ground state contributes. Here you will work at moderate $\beta$, where excited states contribute too, and check the result against an exact answer.

Divide the imaginary time $\beta$ into $N$ slices of width $\delta\tau = \beta/N$ (with $m = \hbar = 1$). A closed path $x = (x_0, x_1, \ldots, x_{N-1})$, with $x_N \equiv x_0$, has probability

$$p(x) \propto e^{-S_E(x)}, \qquad S_E(x) = \sum_{i=0}^{N-1} \left[ \frac{(x_{i+1} - x_i)^2}{2\,\delta\tau} + \delta\tau\, V(x_i) \right], \tag{4}$$

and the thermal average of $x^2$ is the average of $\frac{1}{N}\sum_i x_i^2$ over sampled paths. For the harmonic oscillator, $V(x) = \frac{1}{2}x^2$, the exact result as $\delta\tau \to 0$ is

$$\langle x^2 \rangle_\beta = \frac{1}{2} \coth\left(\frac{\beta}{2}\right). \tag{5}$$

**Problem A [10 points]:** Take $\beta = 1$ and $\delta\tau = 0.1$ ($N = 10$). Sample paths with Metropolis updates of one slice at a time, $x_i \to x_i + u$ with $u$ uniform in $[-h, h]$; one *sweep* is $N$ attempted updates.

(i) Run at least 50,000 sweeps. Choose $h$ so that 40–60% of updates are accepted, and justify your burn-in. Plot the time series of $\frac{1}{N}\sum_i x_i^2$ and its autocorrelation function, and estimate the integrated autocorrelation time $\tau_{\text{int}}$ (in sweeps). Report $\langle x^2 \rangle_\beta$ with an error bar from blocking (Lab 12), and compare it with the naive error bar that treats every sweep as independent.

(ii) Explain why single-slice updates decorrelate slowly. (Hint: how far can one update move the center of the path, $\bar{x} = \frac{1}{N}\sum_i x_i$, when $\delta\tau$ is small?)

**Problem B [10 bonus points]:** *(Optional.)*

(i) Add a second move after every sweep that shifts the whole path, $x_i \to x_i + d$ for all $i$, with $d$ uniform in $[-h', h']$. Which terms of $S_E$ change, and why is this a valid Metropolis move? Repeat A(i) with both moves, and compare $\tau_{\text{int}}$, the error bar, and the agreement with the exact result.

(ii) Using both moves and $\delta\tau = 0.1$, compute $\langle x^2 \rangle_\beta$ with error bars for $\beta = 0.5$, 1, 2, and 4. Plot the results against $\beta$ together with the exact result $\frac{1}{2}\coth(\beta/2)$. At $\beta = 1$, the time discretization with $\delta\tau = 0.1$ lowers $\langle x^2 \rangle_\beta$ by about 0.09%. Compare this with your error bar: is your result limited by statistics or by discretization?

## Autograded results

Have your code write `results.json` and upload it with the source files. Any language can write JSON; a Python example follows the field table.

- **Averages:** average over the sweeps you keep after burn-in, and report how many those are in the `n_sweeps` fields.
- **Error bars:** every `x2_err` field except `x2_err_naive` is a one-standard-deviation error bar on the mean that accounts for autocorrelation, as in Lab 12.
- **Magnetization:** `m` may hold $\langle m \rangle$ or $\langle |m| \rangle$, and either sign is fine below $T_c$.
- **Seed:** seed every random number generator from one fixed integer, such as your TSN, using that integer plus $k$ for your $k$th run, and report it as `seed`. At Oral Checkpoint 3 we may rerun your code with it and compare with your `results.json`.
- **Bonus:** leave out the fields of any part of Problem 2B you skip.

| Field | Contents |
|---|---|
| `tsn` | Your TSN, e.g., `"212345678"` |
| `seed` | The integer your random number generators are seeded from |
| `m` | Problem 1B: the average of $m$ at $T = 1.0, 1.2, \ldots, 4.0$ (16 values) |
| `h` | Problem 2A(i): your hit size $h$ |
| `acceptance` | Problem 2A(i): the fraction of single-slice updates accepted |
| `n_sweeps_single` | Problem 2A(i): the number of sweeps in your averages |
| `x2_single`, `x2_err_single` | Problem 2A(i): $\langle x^2 \rangle_\beta$ and its blocking error bar |
| `x2_err_naive` | Problem 2A(i): the naive error bar (sweeps taken as independent) |
| `tau_single` | Problem 2A(i): $\tau_{\text{int}}$, in sweeps |
| `h_shift` | Problem 2B(i) (bonus): your $h'$ |
| `n_sweeps_both`, `x2_both`, `x2_err_both`, `tau_both` | Problem 2B(i) (bonus): the same quantities as in 2A(i), with both moves |
| `n_sweeps_beta` | Problem 2B(ii) (bonus): the number of sweeps in your averages at each $\beta$ |
| `x2_beta`, `x2_err_beta` | Problem 2B(ii) (bonus): $\langle x^2 \rangle_\beta$ and its error bar for $\beta = 0.5, 1, 2, 4$ |

```
import json

results = {
    "tsn": "212345678",
    "seed": seed,
    "m": [float(v) for v in m_of_T],
    "h": h, "acceptance": float(acceptance), "n_sweeps_single": n_single,
    "x2_single": float(x2_single), "x2_err_single": float(x2_err_single),
    "x2_err_naive": float(x2_err_naive), "tau_single": float(tau_single),
    # Problem 2B (bonus); leave out the fields of any part you skip
    "h_shift": h_shift, "n_sweeps_both": n_both,
    "x2_both": float(x2_both), "x2_err_both": float(x2_err_both),
    "tau_both": float(tau_both),
    "n_sweeps_beta": n_beta,
    "x2_beta": [float(v) for v in x2_beta],
    "x2_err_beta": [float(v) for v in x2_err_beta],
}
with open("results.json", "w") as f:
    json.dump(results, f)
```

## Grading and feedback

- **Autograded points:** 1B: 6, 2A: 7; 2B: 4 bonus. The report earns the remaining points.
- **Tolerances:** your numbers are Monte Carlo estimates, so they are compared with exact results using tolerances set by your error bars and numbers of sweeps. Honest error bars are part of the answer: ones that are too small or too large lose points too.
- **Feedback:** Gradescope immediately checks that it can read `results.json`, your TSN, and your seed. Scores appear after the deadline; resubmit as often as you like before then.